

ARRAYS AND ARRAY OPERATIONS IN SCALA



by
Dr M Rajasekhara Babu

School of
Computer Science and Engineering
<https://www.rajababu.org>



VIT[®]

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Vellore-632014, Tamil Nadu, India



Learning Objectives

By the end of this session, you will be able to:

01

Define & Create

Create arrays in Scala using multiple methods.

02

Access & Modify

Read and update individual array elements.

03

Traverse

Loop through arrays using different techniques.

04

Apply Methods

Use built-in array methods for common operations.

05

Compute

Perform basic array-based calculations like sum and average.

What Is an Array?

- An array is a collection of elements of the same datatype stored together under a single name.
- Instead of declaring separate variables, you group related values efficiently.

One Name

Stores multiple values under a single identifier.

Loop-Friendly

Easy to process all elements using loops.

Memory Efficient

Contiguous storage makes access fast and predictable.

Without Arrays vs. With Arrays

Without Arrays

Each value needs its own variable — hard to manage at scale.

```
val mark1 = 75
```

```
val mark2 = 80
```

```
val mark3 = 85
```

```
val mark4 = 90
```

Using Arrays

All values stored under one name — clean, concise, and scalable.



```
val marks = Array(75, 80, 85, 90)
```

One line replaces four — and scales to hundreds of values effortlessly.

Creating Arrays — Two Methods

Method 1: Direct Initialization

Provide values immediately at declaration.

```
val numbers = Array(10, 20, 30, 40, 50)
```

Method 2: Fixed-Size Array

Declare size first; assign values later. Default value is 0.

```
val arr = new Array[Int](5)  
// Initially: 0, 0, 0, 0, 0
```

```
arr(0) = 100  
arr(1) = 200  
println(arr(0))  
// Output: 100
```

❏ Use Method 1 when values are known upfront. Use Method 2 when size is fixed but values are computed later.

Accessing Array Elements

Array indexing in Scala starts from 0. Use the element's index inside parentheses to access it.

```
val fruits = Array("Apple", "Orange", "Banana")
```

```
println(fruits(0))    // Output: Apple  
println(fruits(2))    // Output: Banana
```

Index 0: Apple

Index 1: Orange

Index 2: Banana

Updating Array Elements

Unlike `val` variables, arrays are mutable — their elements can be changed after creation, even when declared with `val`.

```
val numbers = Array(10, 20, 30)
numbers(1) = 50
println(numbers.mkString(", "))
// Output: 10, 50, 30
```

Original

`Array(10, 20, 30)`
— index 1 holds
value 20.

After Update

`numbers(1) = 50`
replaces 20 with 50.

Result

`mkString(", ")`
prints: 10, 50, 30

Scala offers three clean ways to iterate over every element in an array.

Three Ways to Traverse

1

For-Each Loop

```
for(num <- nums) {  
  println(num)  
}
```

2

Index-Based Loop

```
for(i <- 0 until  
  nums.length) {  
  println(nums(i))  
}
```

3

foreach() Method

```
nums.foreach(println)
```

All three produce the same output for `Array(5, 10, 15, 20)`: **5 → 10 → 15 → 20**. Choose based on whether you need the index.

Built-In Array Methods

Scala arrays come with powerful built-in methods for common computations —



.length

Returns the number of elements.

`arr.length → 3`



.sum

Returns the total of all elements.

`arr.sum → 60`



.max

Returns the largest element.

`arr.max → 30`



.min

Returns the smallest element.

`arr.min → 10`



.sorted

Returns a new sorted array.

`data.sorted → 10,20,40,50`

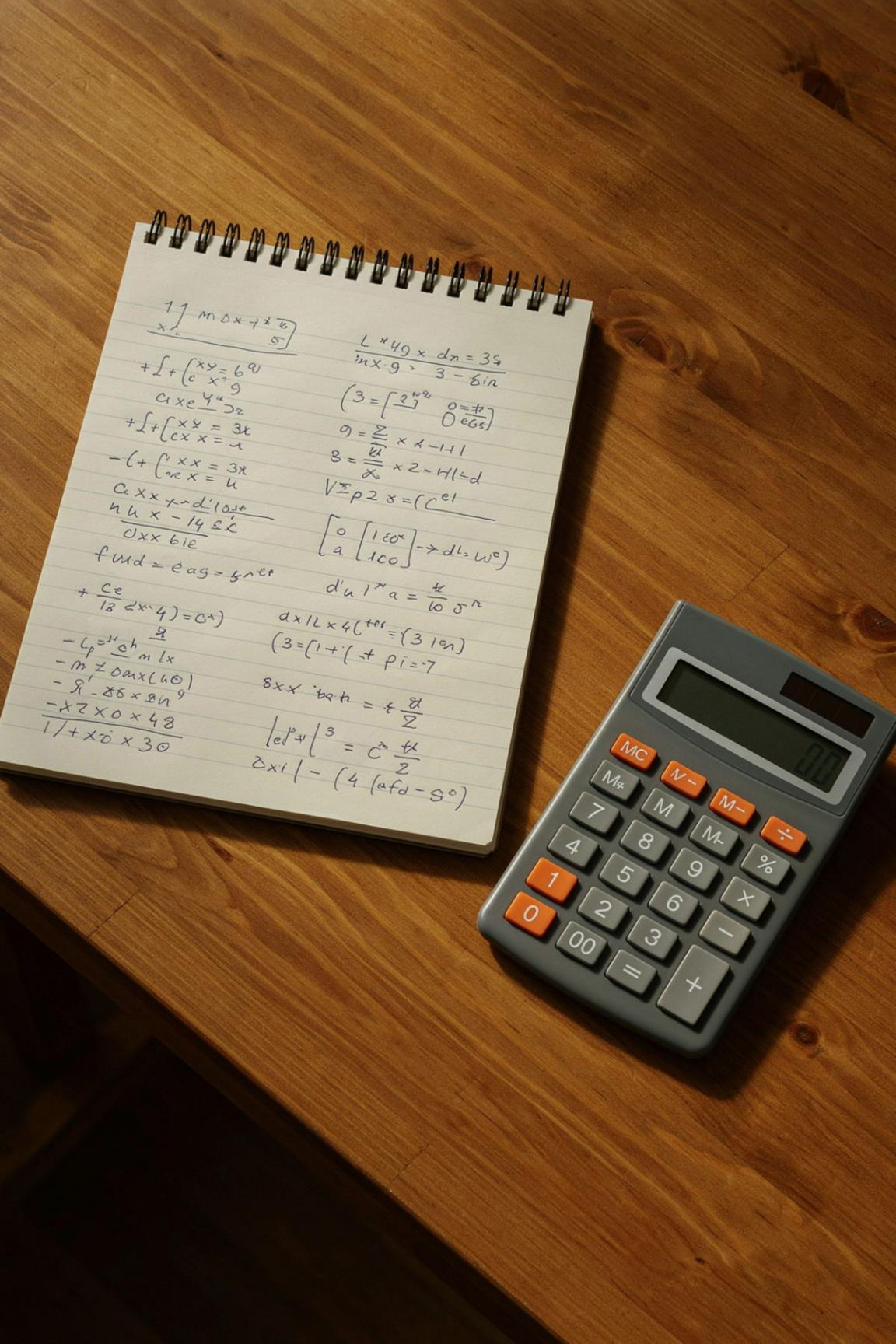


Array Methods — Quick Reference

Applied to `val arr = Array(10, 20, 30)` and `val data = Array(50, 10, 40, 20)`:

Method	Code	Output
Length	<code>arr.length</code>	3
Sum	<code>arr.sum</code>	60
Max	<code>arr.max</code>	30
Min	<code>arr.min</code>	10
Sorted	<code>data.sorted.mkString(",")</code>	10,20,40,50





PROGRAM DEMO

Sum of Array Elements

A complete Scala program that iterates through an array and accumulates a running total.

Demo: Sum Program

```
object ArraySum {  
  def main(args: Array[String]): Unit = {  
    val numbers = Array(10, 20, 30, 40, 50)  
    var sum = 0  
    for(num <- numbers)  
      sum += num  
    println("Sum = " + sum)  
  }  
}
```

Input Array

Array(10, 20, 30,
40, 50)

Loop Logic

Adds each element to
sum on every
iteration.

Output

Sum = 150

Demo: Average Program

Using built-in methods makes computing the average concise and readable.

```
object AverageDemo {  
  def main(args: Array[String]): Unit  
= {  
    val marks = Array(80, 85, 90, 95)  
    val avg = marks.sum.toDouble /  
marks.length  
    println("Average = " + avg)  
  }  
}
```

Output

Average = 87.5

**Sum = 350 ÷ Length = 4 →
87.5**

Key Technique

.toDouble ensures decimal division instead of integer division — critical for accurate averages.

CLASSROOM ACTIVITY

Work through these three activities in sequence to reinforce today's concepts.

1 Display Marks

Create an array containing five subject marks and display each one.

2 Total & Average

Calculate the total and average marks from your array.

3 Highest & Lowest

Find the highest and lowest marks using array methods.

Assignment Problems

1

Sum & Average

Find the sum and average of 10 numbers stored in an array.

2

Max & Min

Find the maximum and minimum element in an array.

3

Even & Odd Count

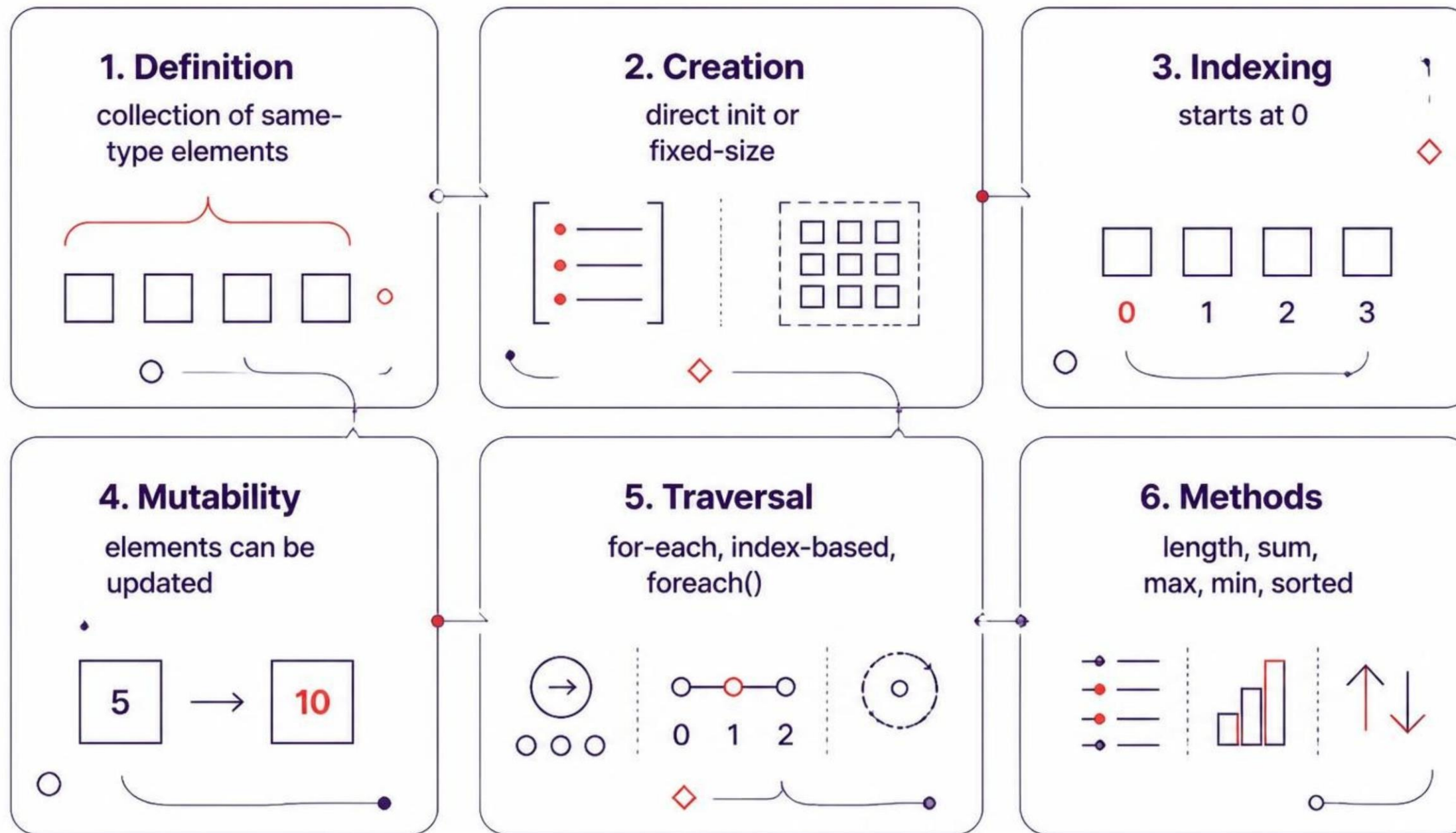
Count the number of even and odd numbers in an array.

4

Search Element

Search for a given element in an array and report whether it exists.

Concept Recap: Arrays at a Glance



Key Takeaways

→ **Arrays group related data**

Store multiple same-type values under one name for cleaner, scalable code.

→ **Indexing starts at 0**

Always access elements using zero-based indices inside parentheses.

→ **Arrays are mutable**

Element values can be updated even when the array is declared with `val`.

→ **Built-in methods save time**

`.sum`, `.max`, `.min`, `.sorted` — use them instead of writing manual loops.